

Preparing the workforce, setting up the infrastructure for adoption, and defining an adoption strategy are critical enablers for MBSE adoption. But once an organization gets to that point, they need to do the actual adopting. A key factor here is training. Many people find MBSE challenging to learn. This could be because they are actually learning multiple things at once: a tool, a modeling language, a method, and system engineering. According to practitioners, all stakeholders need some level of training, but the amount and what they need to learn varies [13].

Table 2 shows four categories of roles and their common MBSE training needs. Different roles may require only a subset of the components of MBSE (i.e., tool, language, method, SE concepts) that they need to be familiar with. For example, someone serving as a ‘model reviewer’ likely does not need training related to the specific tool or method. But they do need to be able to understand and interpret MBSE artifacts. In this case, some training in the modeling language may be all that is required. Two aspects are worth noting. First, the roles are not necessarily mutually exclusive, but a given individual may fulfill more than one role in different capacities at once. Second, whereas some teams may decide to split the role of a modeler and a regular systems engineer, some other teams may assign each systems engineer (or engineer in general) modeling tasks without depending on a dedicated team for modeling tasks.

Integration of work is another critical factor to the successful adoption of MBSE enterprise-wide [15]. Systems engineering in general involves different functional teams/disciplines to come together, and this is still the case with MBSE. Coordination between these two groups is key. A team can create a great model, but if it does not accurately represent the system then it is of little use. Additionally, system models are often broken up into a collection of smaller system models. In other words, multiple teams are responsible for modeling a component of the system that is ultimately integrated together. In these cases, having some level of standardization/consistency across those groups is critical since those disparate models will need to ultimately integrate. There are many different ways to correctly model the same thing, so having some guidelines for groups to follow relative to some of those decisions will help the final product be more cohesive and reduce integration effort.

Adoption of MBSE is difficult because there are many moving parts. But ultimately, there are many ways in which MBSE can provide value to the team. An organization *just* needs to find what that is and use that purpose to navigate through all the different decisions and components that need to be made.

Categories of roles who need training	
Model reviewers	Leaders, stakeholders, or customers who need to know how to use the models to make decisions
Developers (Modelers)	People who are building and maintaining the models in the tools, so they will need detailed knowledge of how the tool works
Other engineers	People who will be working in the model to some capacity. These people are often senior engineers or people from other disciplines who are helping with the content of the model
Administrators	People who work in IT or software who will be managing the relevant accounts, licenses, tools, etc.

Table 2. Categories of roles requiring MBSE training

7. CONCLUSIONS

Given the speed at which the world keeps accelerating, transitioning to model-based/digital approaches to engineering is inevitable. Traditional practices simply cannot keep up with the demand for rapid development and production of ideas into reality. Systems engineering itself is not changing; just the way it is done is. The key to embracing MBSE is to focus on good systems engineering and recognize that model-based is largely using computer-aided techniques to better execute what systems engineers have always done.

MBSE is gaining a foothold in industry and government as more and more people are starting to see the benefits of using it. At its core, MBSE is about capturing data in a better way using modern techniques enabling teams to better understand, communicate, reason, and retain knowledge. The cost of adoption in terms of time and money is high, but it is argued that the cost of not adopting it will be even higher in the long run.

BIBLIOGRAPHY

1. Blanchard, B.S. and J.E. Blyler, Systems Engineering Management. 5th ed. 2016, Hoboken, NJ, USA: John Wiley & Sons, Inc.
2. INCOSE, Systems Engineering Vision 2025. 2014.
3. INCOSE, Systems Engineering Vision 2035. 2023.
4. Defense, D.o., DoD Modeling and Simulation (M&S) Glossary. 1998.
5. Henderson, K. and A. Salado, Is CAD A Good Paradigm for MBSE? INCOSE International Symposium, 2021. 31(1): p. 144-157.
6. Long, D. and Z. Scott, A Primer For Model-Based Systems Engineering. 2nd ed. 2011, USA: Vitech Corporation.
7. Cratsley, B., et al., Interpretation Dscrepancies of SysML State Machine: An Initial Investigation, in Conference on Systems Engineering Research. 2020: Virtual.
8. Henderson, K. and A. Salado, Value and benefits of model-based systems engineering (MBSE): Evidence from the literature. Systems Engineering, 2021. 24(1): p. 51-66.
9. Blackburn, M. and T. West, Fundamentals of Digital Engineering, in Systems Engineering for the Digital Age. Practitioner Perspectives, D. Verma, Editor. 2023, John Wiley and Sons, Inc.: Hoboken, NJ, USA. p. 3-24.
10. Salado, A., 5.5.2 Efficient and Effective Systems Integration and Verification Planning Using a Model-Centric Environment. INCOSE International Symposium, 2013. 23(1): p. 1159-1173.
11. Dunbar, D., et al., Transforming Systems Engineering Through Integrating Modeling and Simulation and the Digital Thread, in Systems Engineering for the Digital Age. Practitioner Perspectives, D. Verma, Editor. 2023, John Wiley and Sons, Inc.: Hoboken, NJ, USA. p. 47-68.
12. Dunbar, D., et al., Driving digital engineering integration and interoperability through semantic integration of models with ontologies. Systems Engineering, 2023. 26(4): p. 365-378.
13. Henderson, K., T. McDermott, and A. Salado, MBSE adoption experiences in organizations: Lessons learned. Systems Engineering, 2024. 27(1): p. 214-239.
14. Henderson, K., et al., Towards Developing Metrics to Evaluate Digital Engineering. Systems Engineering, 2023. 26: p. 3-31.
15. Henderson, K., Exploring the Adoption Process of MBSE: A Closer Look at Contributing Organizational Structure Factors, in Grado Department of Industrial and Systems Engineering. 2022, Virginia Tech: Blacksburg, VA, USA.
16. Henderson, K. and A. Salado, The Effects of Organizational Structure on MBSE Adoption in Industry: Insights from Practitioners. Engineering Management Journal, 2024. 36(1): p. 117-143.
17. Weilkiens, T., Adoption of MBSE in an Organization, in Handbook of Model-Based Systems Engineering, A.M. Madni, N. Augustine, and M. Sievers, Editors. 2020, Springer International Publishing: Cham. p. 1-19.

DAVID LONG

David Long is the President of Blue Holon and a Research Scientist with the Systems Engineering Research Center (SERC). For over 30 years, he has helped organizations around the world increase their systems engineering proficiency while simultaneously working to advance the state of the art. He works with government and commercial



organizations as they assess, adopt, and deploy new methods and tools to enhance their engineering enterprise. Throughout his career, David has played a key technical and leadership role in advancing and expanding the practice of systems engineering. David founded and led Vitech where he developed innovative, industry-leading methods and software (CORE™ and GENESYS™) to engineer next-generation systems. David is a frequent presenter at industry events worldwide delivering keynotes and workshops spanning the foundations of systems engineering, practical model-based systems engineering, digital engineering, and the future of engineering systems. His experiences and efforts led him to co-author the book *A Primer for Model-Based Systems Engineering* to spread the fundamental concepts of this key approach to modern challenges. An INCOSE Fellow and Expert Systems Engineering Professional (ESEP), David was the 2014/2015 President of INCOSE. David currently serves as INCOSE's Director for Strategic Integration and as a coach in INCOSE's Technical Leadership Institute. David holds a BS in Engineering Science and Mechanics and an MS in Systems Engineering from Virginia Tech.

DR. KAITLIN HENDERSON

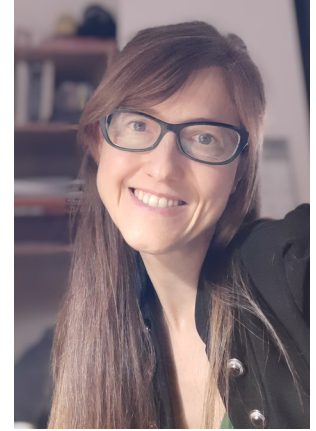
Dr. Kaitlin Henderson is a Systems Architect at Radiance Technologies, where she supports various SysML modeling efforts. She has a PhD in Industrial and Systems Engineering with a concentration in Management Systems from Virginia Tech in Blacksburg, Virginia. She also earned her Bachelor's and Master's degrees in



Industrial and Systems Engineering at Virginia Tech. Kaitlin has contributed to the fields of Model-Based Systems Engineering (MBSE) and Digital Engineering (DE) with her works on MBSE adoption, MBSE domain maturity, and MBSE/DE performance measurement. Kaitlin won the Best Presentation Award at the SERC Doctoral Student Forum in 2020 and has been recognized for her 2021 and 2022 publications in Systems Engineering.

BELINDA MISIEGO TEJEDA

Belinda Misiego Tejeda is the chief of innovation at Isdefe and coordinates Systems Engineering Campus in the company. Through this Campus, internal training courses are carried out with its own personnel in the different areas of this discipline: systems engineering initiation, fundamentals of systems engineering and various



specific courses. She has more than 20 years of professional experience in the security and defense sectors, where she has developed and managed different systems engineering projects, mainly in the field of sensors and electronic warfare. Belinda, member of INCOSE and systems engineering professional (CSEP), was part of the board of the Spanish Association in Systems Engineering (AEIS), the Spanish chapter of INCOSE between 2017 and 2022. In addition, she served as Secretary of the EMEA Director of INCOSE in 2019-2022. Belinda has a BS/MS in Electrical and Computer Engineering and a graduate certificate in Research and Market Techniques from the University of Valladolid and a MA in Contract and Program Management in the Public Sector from the Spanish University of Distance Education (UNED).



"Is the universe really so loosely coupled? Or is this small dimensionality due to the fact that the humans who developed the equations controlled their experiments in accordance with their cognitive limitations?"

G. Friedman

Digital Transformation in System Development

Christopher L. Delp, *Jet Propulsion Laboratory, California Institute of Technology* (christopher.l.delp@jpl.nasa.gov)

Dr. Joe Gregory, *The University of Arizona* (joegregory@arizona.edu)

Luis Miguel Aparicio Ortega, *Isdefe* (laparicio@isdefe.es)

Abstract

This chapter presents the novel capabilities enabled by digital models and high computational power of current workstations, cloud, and grid computing to support systems development, integration, and qualification. It introduces the concept of digital transformation and explains how it builds on the processes of digitization and digitalization. The key technologies that enable digital transformation are presented and discussed. These include formal languages and semantic web technologies. The chapter then looks at some examples of how digital transformation can lead to improved outcomes with regards to systems engineering practice. Examples include enhanced traceability, the automated generation and evaluation of architectures, set-based design, and the integration of physics-based models into systems engineering models.

Keywords:

Digital engineering, digital thread, tradespace exploration, set-based design, Multidisciplinary Design Optimization (MDO), semantic web technologies, ontologies, systems model.



1. INTRODUCTION

Interest in digital transformation has seen rapid growth in the last decade. In this chapter, we present an overview of this rapidly evolving field and discuss some of those relevant, emerging technologies that digital transformation aims to leverage and provide examples of how they are being applied to Systems Engineering (SE). First, to understand what we mean by Digital Transformation, there is a crucial distinction that needs to be made between two commonly used terms: digitization and digitalization.

Digitization refers simply to the change that occurs when an analog artifact, whatever it may be, is transformed into a digital artifact. If we take a physical document, such as a report or a photograph, and convert it into a digital format through scanning or simply by typing text in a word processor, we have performed digitization. This kind of process has been taking place for decades, and probably everyone understands the benefits of working with digital artifacts instead of with tons of paper. For example, digitization of information enabled vast quantities of information to be transmitted digitally and without loss. This process of digitization was enabled by advances in computer miniaturization and networking infrastructure, and it revolutionized the ways in which information can be managed and communicated, known as the Information Age or the Third Industrial Revolution.

Digital transformation, however, goes beyond digitization. In digitalization, engineers work with models and data that are digitally linked. Any information consumption is just a visualization of the underlying models and data. In other words, the information and its visualization are decoupled.

Because of this, digitalization often involves rethinking traditional processes to take full advantage of the capabilities that have been made possible by connecting data and models in a machine-readable manner. Clearly, digitalization involves some deep thinking about how modern technologies can be effectively leveraged, particularly when applied within a business context. When it is to be applied within the boundary of an organization, digitalization requires careful consideration of the necessary cultural and organizational changes, relevant talent acquisition and development, data governance and security within and across organizational boundaries, as well as multiple other aspects. Together, this process of digitization and digitalization to improve processes within a business context is known as the Digital Transformation [1].

Before we look at some of the technologies that enable digital transformation, let us consider what an ideal 'digitalized' SE process might look like. In other words, to understand what is required to support our digital transformation, we must have a vision of what our digitally transformed enterprise should be capable of. In a nutshell, the information generated from all stages of the system lifecycle is in digital form, is linked through a data-driven architecture of shared resources, feeds descriptive and quantitative models, and can be used for real-time and long-term decision-making" [2] (ref. Figure 1). Practically speaking, this means that data can be seamlessly exchanged between the different tools that support modeling in each of system lifecycle phases. Not only does this provide the necessary traceability between design decisions, tests, requirements, and so on; it enables decision support analyses that were previously infeasible.

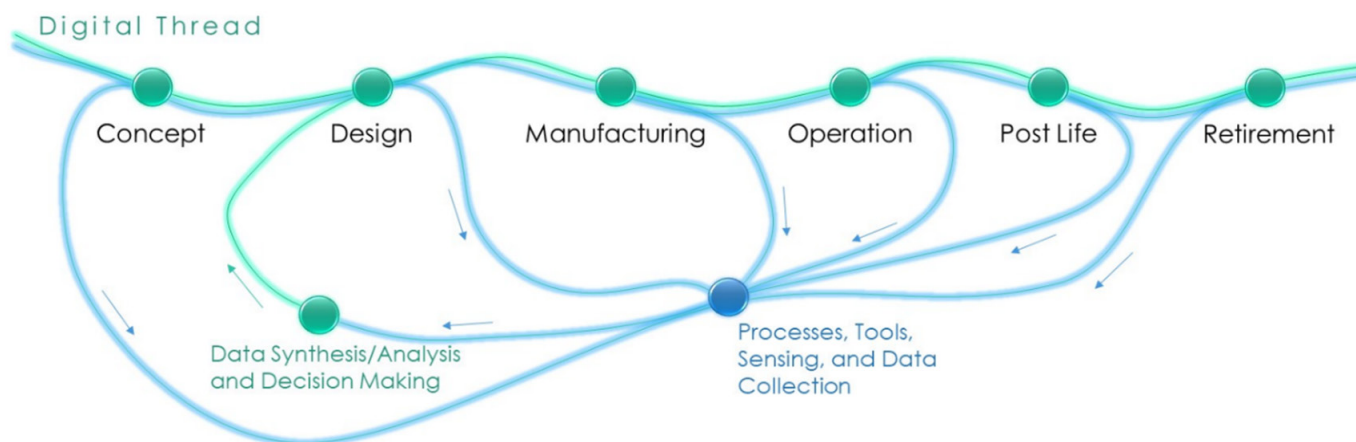


Figure 1. The Digital Thread related to Systems Engineering [adapted from [2]]

Furthermore, this capability extends beyond the processes of any single organization – an ideal digital thread enables collaboration and seamless data exchange across organizational boundaries to support critical aspects such as supply chain integration. Within the confines of appropriate security and privacy protocols, data may be rapidly accessible across the digital thread to support decision-making at the enterprise level. Appropriate change propagation pipelines ensure that downstream effects of a change to the data are identified and acknowledged.

2. ENABLERS OF DIGITAL TRANSFORMATION

2.1. Computer languages unleash the power of applied mathematics

It is the thrust of increasingly sophisticated systems coupled with the critical impacts of quality and cost that drive the need for systems engineering practices that are quantifiable and analyzable. Achieving this need often requires the development of an executable representation of the system. This intersection of need and the availability of a particular kind of system – the modern computational environment – is a rather poetic result: it is now possible to use a human-developed system to enter a recursive development cycle

whereby improvements in the discipline of SE can deliver improvements in how we practice the discipline.

The core of this concept is rooted in the ability to mathematically model the world around us. In Computer Science, the essence of this innovation is the weaving together of computational instruction with what is arguably the most significant invention in human history – language.

Since its inception, systems engineering has focused on the specification of the systematic behavior of an element known as a ‘System’. This element can take many forms as long as the central concept defining its existence is systematic behavior. Traditional SE focused on informal descriptions, human thought experiments, and heuristics to tackle the increasingly complex problems that SE was expected to deal with [3]. This approach often relied on expensive testing on real-world builds and rigid inflexible constraints on changes to meet unforeseen challenges.

The widespread development, deployment, and adoption of suitable modeling languages to support SE has been described as ‘The Systems Engineering Challenge’ [3]. In recent decades, increased computational power and the development of a multitude of declarative modeling languages have contributed to the advancement of SE. Some of these are displayed in Figure 2.

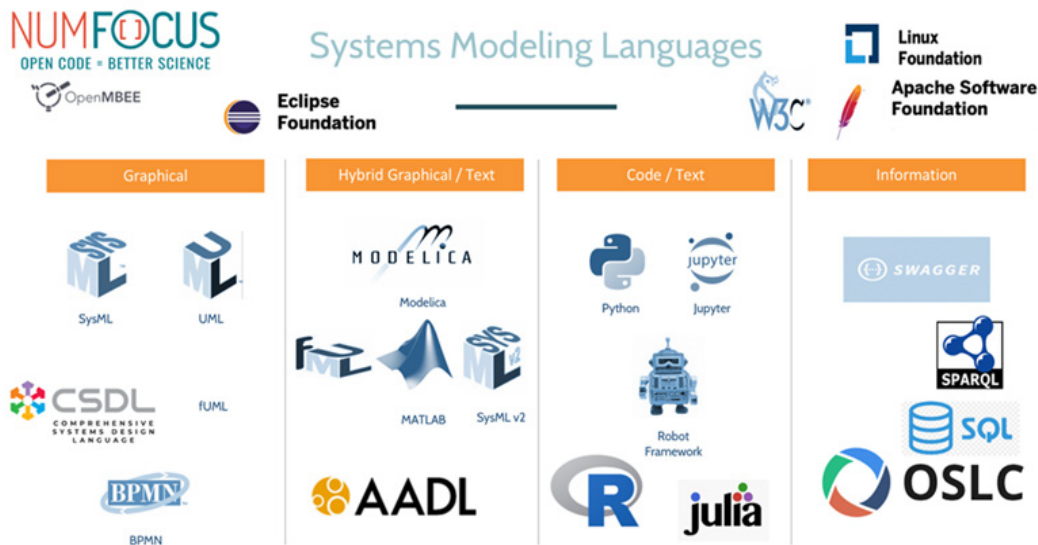


Figure 2. Examples of declarative modeling languages to support systems engineering and some of the standards bodies and open source software foundations that sustain them

Graphical languages such as SysML version 1 and CSDL provide the capability to visually model specifications of concepts and relationships. Text/code-based languages such as Julia and Robot expand the power across the lifecycle to capture detailed execution, analysis, and systems testing capability. The new generation of hybrid languages such as AADL, SysML version 2, or Modelica point to a powerful capability to specify, visualize, code, and analyze systems across the lifecycle. Information modeling languages such as OWL/SparQL and OpenAPI/Swagger also play a crucial role in modern systems modeling and Digital Engineering (DE). They provide a foundation for supporting the web-based interoperability required for model integration and interchange.

The languages presented in Figure 2 can all be considered ‘Declarative Languages’. Declarative languages focus on semantics capable of being solved in a variety of ways besides traditional execution. Often, they can be solved using analytic techniques as opposed to procedural semantics which execute statements in the order they are read. Declarative models tend to have mathematical statements that can be solved in more than one way. It is worth noting as well that while languages such as MATLAB, Python, or Julia are not strictly declarative, they build libraries and capabilities used heavily in engineering that perform analysis on declarative models such as algebra solvers and other equation-based analysis. Examples include First-Order Logic (FOL) and Finite State Machine (FSM). FOL in particular has received significant attention, and not just in engineering, due to its ability to precisely capture knowledge and infer new knowledge based on a set of rules [5]. FSMs are a good example of this type of language. They declare states, behavior, and events. The a-causal nature of an FSM can be interpreted differently while still retaining the mathematical formality of the FSM. The most direct way to solve an FSM is to define a trajectory of events and test the FSM to see the output. A less obvious example would be to use a solver to analyze the entire FSM to find out if the event trajectory is valid. A third example would be to analyze the FSM to determine if there are any states defined such that they can never be reached.

While declarative languages tend to enable formal methods in DE, there is still value in informal descriptive and qualitative representations such as narrative text, graphics, diagrams, and pseudo code. They are an important part of the development cycle. As models are developed against informal material they undergo a process called the Model-Hardening Process. This process captures the relationships between the qualitative representations and the quantitative representations as the design matures.

With all the right ingredients, SE can produce precise, accurate, and complete digital representations of systems and the processes that will realize them. The ‘right ingredients’ are those that leverage the power of computation and the precision of formal languages to manage the huge amount of data that is associated with an SE project, automate SE processes through inference and reasoning, and preserve rigor within the system lifecycle. One area we can look to for inspiration in this regard is an area that has witnessed huge progress in the last 30 years in terms of information management and retrieval: the evolution of the World Wide Web.

2.2. Semantic web technologies

The Semantic Web is an extension of the World Wide Web that intends to make data machine-readable and provide a standard structure for data representation and reasoning [6]. At the core of this evolution is the transition from a web that “consists largely of documents for humans to read to one that includes data and information for computers to manipulate” [7]. An obvious first example might be the evolution of the search engine. Far from being a simple keyword-match web-crawler algorithm, modern search engines exploit this hyper-connectivity between data to understand the meaning behind your search query, employ complex algorithms utilizing machine-learning to understand user intent, and often generate a natural-language personalized response.

Semantic web technologies are the technologies that enable these capabilities. They provide standardized ways to represent data in triples of subject-predicate-object format that allows to describe knowledge such as ‘Curiosity is a Mars Rover’ or ‘Curiosity executes Analyze Soil’. In this way, it is possible to build up complex networks of interconnected data that are entirely built on this simple triple pattern. By using an ontology, these networks can be leveraged to check data validity, as well as to infer new information that we have not explicitly declared. For example, consider the ontology in Figure 2. In this ontology, three classes are defined: System, Function, and Mars Rover. It has also been stated that Mars Rover is a subclass of System. An object property (a relation between two classes) called ‘executes’ specifies that the domain can only be a System, and the range can only be a Function. This ontology can be used to validate the dataset presented earlier (i.e., ‘Curiosity is a Mars Rover’ and ‘Curiosity executes function “Analyze Soil”’), and to infer new information that that was not previously stated. Because Curiosity is a Mars Rover, and the ontology specifies that Mars Rover is a subclass of System, it is possible to infer that

Curiosity is also a System. Also, because Curiosity (a System) executes 'Analyze Soil', it can be inferred that 'Analyze Soil' is a Function. The validity of the dataset is confirmed because no rules in the ontology have been broken. If we had stated that 'Analyze Soil' executed 'Curiosity', our dataset would have been declared inconsistent, as the range of the relation 'executes' can only ever be a Function.

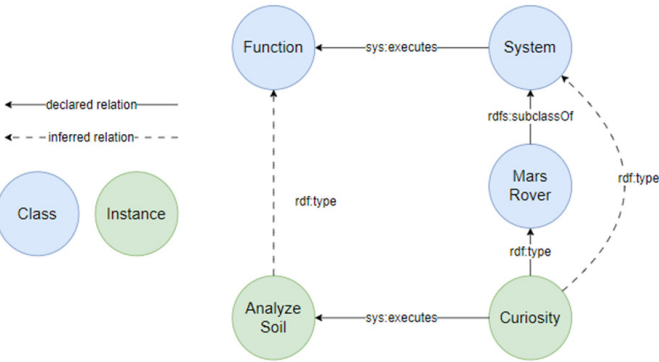


Figure 3. Example Knowledge Graph [Legend: rdf: Resource Description Framework; rdfs: Resource Description Framework Schema; sys: System Ontology]

As an example, Figure 4 shows a partial representation of the ontology to support the modeling of the harness design for a spacecraft.

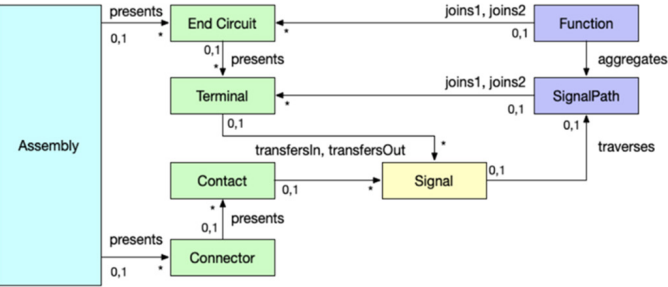


Figure 4. Partial Representation of Harness Design Ontology, from [8]

3. MODEL-BASED LIFECYCLE MANAGEMENT AND QUALIFICATION

In this section, we focus on how modern approaches to DE can support lifecycle management and qualification, e.g. verification and validation and certification. We present examples from the literature regarding traceability across design artifacts, early Verification and Validation (V&V), and model-based reviews.

3.1. Traceability of system design artifacts

Traceability refers to the ability to establish explicit relationships between elements of the system design (e.g., a quantity value, a design decision) across the system lifecycle. In systems engineering, design traceability is a common example, whereby all system elements, design decisions, and test cases can be traced back to a particular requirement (or set of requirements). The purpose of this is to be able to ensure that the results of the design can be explained and checked for review and audit.

DE, and particularly the digital thread, enables unprecedented traceability across the system lifecycle. One of the main benefits of this approach is that data can be defined once in an Authoritative Source of Truth (ASOT), and then distributed across the digital thread to other models and artifacts that require access to the data. An ASOT is the source of a baselined version of data and should be accessible to authorized applications that intend to use the data in some analysis or decision, for example. In this way, engineers can be confident that applications across the entire digital thread (thus representing the entire system lifecycle) are using a valid set of data. The establishment of ASOTs within a digital thread is an effective way to avoid inconsistencies.

Similarly, traceability within a digital thread enables the identification of change propagation paths. If we consider a change to a system requirement, for example, traceability across the digital thread enables engineers to identify the system elements and tests that will be affected, among others.

To achieve this degree of traceability, there are two aspects to consider: data interoperability and technical interoperability [9]. Data interoperability ensures that there is a consistent understanding of the relevant terminology, and a standard data structure to which data can be mapped. Technical interoperability solves the problem of getting data from multiple sources into the same database in the first place – or at least provides point-to-point connections between relevant tools. Technical interoperability can be streamlined by using established integration protocols such as REST API. The Open Services for Lifecycle Collaboration (OSLC) is a community that develops and releases open-course standards aimed at improving integration through REST APIs [10].

The Dragon Architecture, under development by NASA JPL, is a DE environment that comprises multiple systems and software engineering applications and implements the digital thread by connecting the tools together in a graph-

oriented structure, emphasizing explicit relationships to form an integrated heterogeneous model [11] (ref. Figure 5). The use of ASOTs and technical interoperability are core aspects of the Dragon Architecture. It enables the user to define inter-tool relations, thus providing connectivity that integrates requirements, architecture, detailed HW and SW design, test cases, and so on. The Dragon Architecture achieves this through the application of Internet and World Wide Web based interoperability technology and standards. Interoperability through these technologies and standards is a fundamental principle of the Dragon Architecture.

Although not always necessary, ontologies can help with data interoperability. Interoperability in Dragon, however, does require the expression of semantics in order to integrate and inter operate especially focusing relationships. It is often the case that Ontology is associated with the Semantic Web and the Web Ontology Language. While this is certainly a valid means of modeling an Ontology, it is not the only way

an Ontology can be specified. The application of ontologies to SE aims to “enable more automated sharing of information directly between models to ensure model consistency, improve the rigor of engineering process, and ultimately, reduce the effort needed to get a clear answer to engineering questions” [8]. The Digital Engineering Factory (DEF), for example, is a DE environment under development at the University of Arizona that enables users to integrate data from multiple tools and structures this data in accordance with the University of Arizona Ontology Stack (UAOS) [9]. An ontology stack is a structured hierarchy of ontologies, based on the same Top-Level Ontology (TLO), to support data interoperability between domains. The DEF uses a hub-and-spoke architecture to integrate data from multiple tools into a central database (hosted in the Violet tool). This enables users to define inter-tool connections, thus supporting traceability between requirements, system architecture, detailed HW and SW design, verification activities, and project management (ref. Figure 6).

Dragon Framework

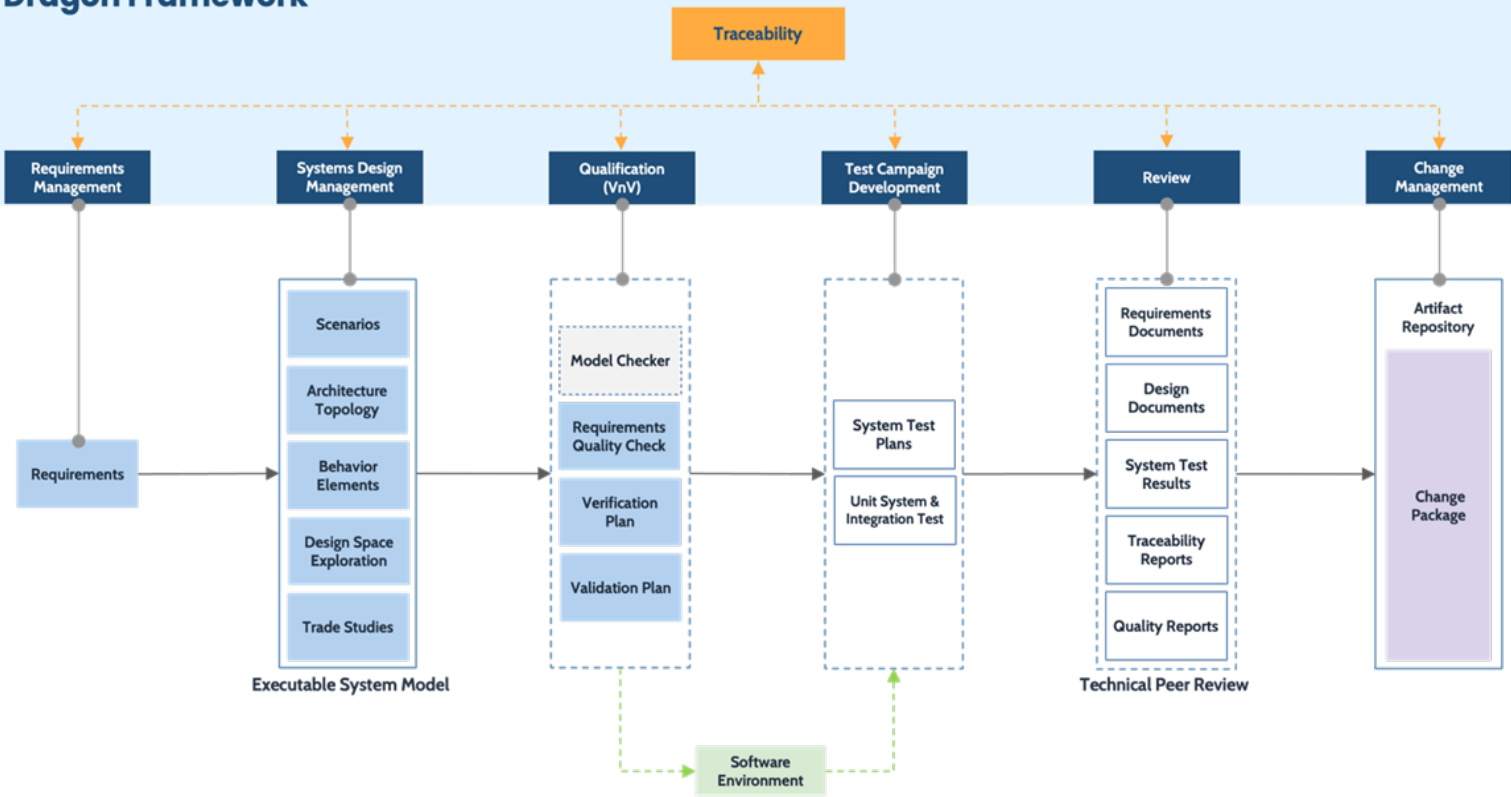


Figure 5. Dragon Architecture Functional View. This is an illustration intended to communicate the general flow of work and conceptual dependencies. It is not intended to describe a process or gates. All functions are realizable with off-the-shelf technology except for model checking

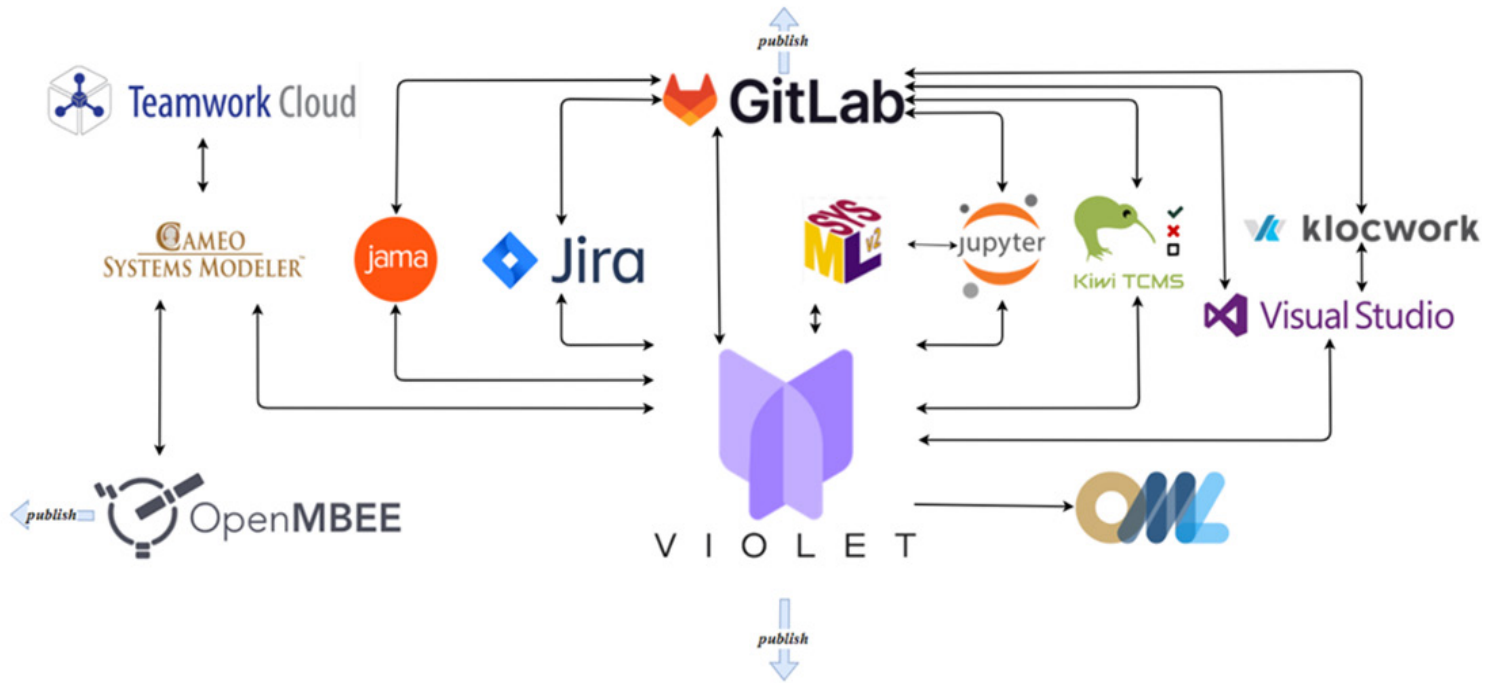


Figure 6. The Digital Engineering Factory (DEF), adapted from [12]

3.2. Early verification and validation

The potential connectivity offered by the digital thread and its enabling technologies also makes it possible to move V&V earlier in the lifecycle [16]. For example, modeling the system architecture in the early stages of design with an executable modeling language enables the execution of simulations and resulting quantitative analyses to assess compliance to requirements [15]. This is valuable because “finding problems early is a key enabler to DE providing full potential value” [13]. Several modeling languages, but not all, provide these executable capabilities. For example, in SysML, the behavior captured in activity and state machine diagrams can be simulated.

In addition to the use of formal methods to execute and simulate models, which involve a complete, mathematically grounded representation of a system, lightweight formal methods can also contribute to early V&V. Lightweight formal methods involve the mathematical representation of some part of the system specification. In this way, lightweight formal methods provide some of the benefits of formal methods, such as the ability to detect errors in the early stages of system development, but without the need to redefine the entire system specification [20].

“One of the strengths of using DE approaches to concept design is that the various models, tools, and ASOTs are all connected via a digital thread. They share the same consistent, authoritative data. During concept design, one useful application of this digital thread is to connect simulation models with system definition models...and unambiguously share requirements, requirements traceability, and system behaviors” [21].

3.3. Model-based reviews and digital signoffs

The digitalization of SE processes also extends to the major gates of the system lifecycle: technical reviews. Traditional document-based review processes have been used by systems engineers for many years to determine whether system development programs can progress to the next stages in their development. However, this traditional approach to technical reviews often leads to “lengthy evaluations of static, contractually obligated documents” that “represent snapshots of the systems as seen through the prism of the entrance criteria [to the next phase of the lifecycle], and do not represent a view of the system in its totality” [22]. Not only is this process inefficient, but it also often lacks the ability to holistically represent the necessary system-related information to stakeholders.

Model-based reviews have the potential to offer improvements in this regard (e.g., [24, 25]). First, reviewers can approve subsets of data/information instead of having to review and approve monolithic aggregations of information in disparate and often overlapping documents. This has been coined as digital signoffs, where even required signoffs can be assigned to different pieces of data for them to be considered baselined, accepted, and/or approved [24]. Second, reviewers can interact with the interoperable ontology stack to find the information that they consider relevant for the review, as opposed to having to dig and hunt for treasure in static documentation. For example, a reviewer might query the DE repository to identify all verification evidence related (i.e., traced) with a requirement, identify all functions that participate in a capability related to a specific need of a stakeholder, or identify the remaining integration and test activities for a specific component. In both cases, it is easy to realize how the use of DE to support technical reviews can result in an accelerated and more fluid process, while potentially improving its efficiency and reducing the likelihood of review gaps.

It should be noted though that model-based reviews come with challenges as well. Among others, models need to be comprehensive, not just limited to traditional 3D drawings; there is a significant learning curve, as the people involved do not only need to learn the new modeling languages and tools but also the new processes to operate in a model-based review and rely on the ability of the reviewers to identify the necessary information in the digital repository [23]. Adopting DE can help to mitigate some of these challenges. Data and technical interoperability provide engineers with a common data structure and database from which project information can be retrieved. Semantic web technologies can be leveraged to automatically validate this dataset and queries to retrieve, and conveniently present, key information can be standardized and automated (e.g., in an autogenerated document or dashboard).

4. BEYOND TRACEABILITY AND LIFECYCLE MANAGEMENT

So far, we have discussed how DE can be used to enhance traceability, enable early V&V, and support the model-based review process. However, DE is not only about traceability and management. In this section, we provide examples of how DE can support a set of unprecedented analysis that have been infeasible in document-based practices. They include tradespace exploration, set-based design, multi-

disciplinary optimization, the integration of physics models within system models, and human-system integration.

4.1. The Tradespace Exploration paradigm

When developing a new system, it is crucial to focus efforts during the early stages of the life cycle on finding a preferred design solution. Since evaluating the entire set of possible solutions requires a significant investment, this analysis is not usually extensive due to the unavailability of the necessary resources. As a result, hasty and generally unjustified decisions are made to reduce the design options and only a few alternatives are evaluated. Consequently, alternatives that could be more valuable than the chosen ones are frequently overlooked.

Tradespace Exploration is a method that leverages digital models and the high computational power of current computers to enable a rapid and comprehensive in-depth analysis of the solution space [26]. Through this strategy, multiple design options are explored as candidate architectures and analyzed to determine the most suitable one. In essence, instead of focusing on finding specific design options, the engineer defines a generic model of the system that can be automatically instantiated as the enumeration of different components and values that their main variables can take. Each resulting solution is evaluated against a set of decision criteria. Among them, a Pareto Front, formed by those solutions that dominate the solution space (that is, there is no solution that performs better in all criteria) can be identified for choosing the desired solution within the set.

For illustrative purposes, consider the design of a new tank with the following desired needs: high transport, high tank protection, and fair mobility. The generic architecture of the tank includes Armor, Weapon System, and Propulsion as its main systems, with each having different design options (Armor: Conventional steel, Ceramics, or Composite; Weapon System: 120 mm cannon or Guided missile launcher; Propulsion: Hybrid electric motor or Internal Combustion Engine). Furthermore, each instantiation can have different performance values, such as having different masses. Considering mass as a surrogate for the overall performance of the different variants of the different technologies (in this case, a total of 71 lower level components), it is possible to enumerate all possible instantiated architectures (e.g., Conventional Steel – Performance 1/120 mm Cannon Performance 1/Hybrid Electric Motor Performance 1,

Conventional Steel – Performance 1/120 mm Cannon
 Performance 1/Hybrid Electric Motor Performance 2,
 Conventional Steel – Performance 1/120 mm Cannon
 Performance 1/Hybrid Electric Motor Performance 3, ... for a
 total of 3,408 architectures) and plot them according to their
 cost (the higher the cost the less preferred the alternative is)
 and value provided in operations (the higher the value the
 more preferred the alternative is; note that both have been
 calculated using notional models), identifying then the Pareto
 Front (ref. Figure 7, where each apparent line is actually the
 effect of many points close together).

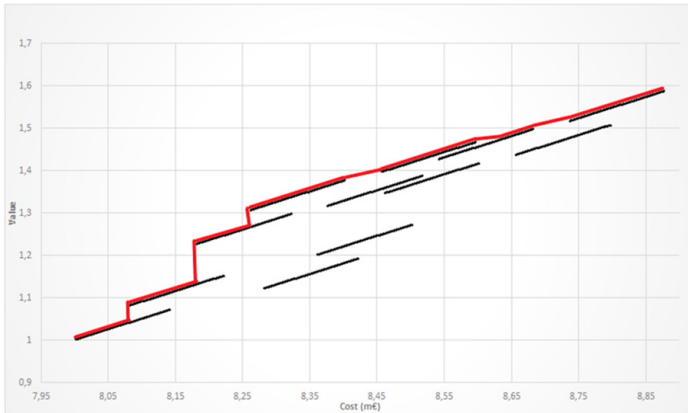


Figure 7. Solution Space for Tank Example relating Cost and Value (Pareto front highlighted)

4.2. The Set-Based Design paradigm

Traditionally, the Point Design Method (PDM) has been employed for system design. It begins with an analysis of alternatives, resulting in the selection of a single concept, which is refined as the development progresses. When a problem is encountered during development, the solution is modified as necessary, generally resulting in higher costs and delays the later the modification is accomplished.

In contrast to this rigid approach, the Set-Based Design (SBD) paradigm encourages maintaining multiple options open and evaluating the trade-offs and advantages of each design rather than quickly converging on a single design [30–32]. It facilitates thorough exploration of the design space and promotes innovation by deferring commitment to a single concept. The solution space progressively narrows as more knowledge and information about the system are acquired

(ref. Figure 8). This reduces the risk of erroneous theories and assumptions of decisions early in the design process, which is delayed for critical decisions until sufficient information and knowledge is available to gain flexibility. Furthermore, as a byproduct of working with sets of solutions instead of with single solutions, SBD provides greater flexibility to adapt to unforeseen circumstances. The use of powerful computers with high computing capacity, and object-oriented modeling, play a significant role in simultaneously developing and evaluating various design concepts at a relatively low cost.

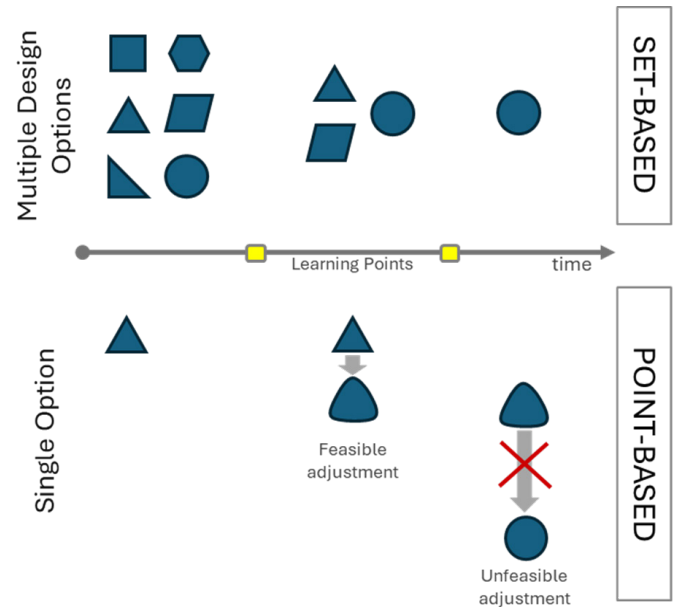


Figure 8. Set-based Design Maturation Process

An SBD essential process consists of three main effects (ref. Figure 9):

1. Definition of the design space, identifying a set of viable alternatives to be progressively developed independently, even by different engineering groups, thereby generating concepts from diverse perspectives. It is crucial to consider a large number of alternatives for evaluation.
2. Discover intersections among different sets of independent solutions to define the core of the new design progressively.
3. Gradually eliminate concepts that prove incompatible or contribute limited value.

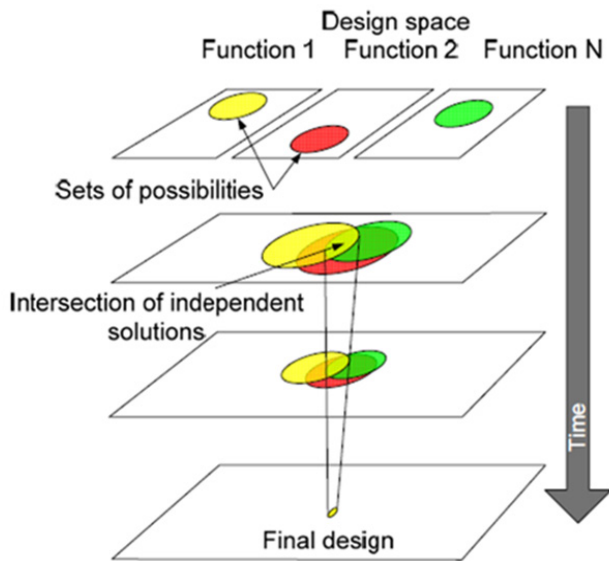


Figure 9. Discovering Intersections in Set-Based Design process [adapted from [33]]

4.3. The MDO paradigm

Traditionally, system decomposition is performed by manually fixing characteristics of the components that form the system and iterating between them until their integration leads to a satisficing system solution. This is due to the coupling that generally exists between the different components of a system. For example, a team designing a satellite may start by estimating the electrical power consumption of each component. They would then aggregate the power dissipated in the different structural panels of the satellite and use that to inform the design of the thermal system (for dissipation). Sizing of the thermal system may feedback into the sizing of the structural panels, and so forth. This iterative process is generally performed manually: propagating the different modifications until something seems promising to work.

Instead, in Multidisciplinary Design Optimization (MDO) the goal is to find an optimal combination of components by managing conflicting objectives not just a satisficing one [34]. MDO leverages computer models to execute optimization algorithms. Particularly, the method consists of integrating mathematical models of the components and executing a global optimization function at the system level. In this way, the design of the components would not lead to just a satisficing solution but to an optimal system. The work of the engineering team moves from iterating their coupled

characteristics to developing accurate models of their components (and their integration), so that the computer can handle the iterations on its own.

MDO does not only identifies a solution that is optimal, but it does so more efficiently than human iteration and guarantees alignment between the decisions at component level and the objectives of the project at the system level. However, one should note that robustness of the solution becomes important, as the accuracy of the underlying models to perform the optimization is likely coarse at the stage when MDO is employed.

4.4. Connecting system models with physics-based models

The connection of system models with physics-based models is a common capability leveraged within a digital thread. In essence, behavioral and structural models of a system architecture can be connected to their design instantiations in physics-based models, propagating constraints (in one direction) and results (in the other direction) between the different models automatically, guaranteeing data consistency and trustworthiness [11].

For example, Figure 10 shows the use of the Dragon Architecture to support the architecture and design of a Rover. Adopting the application of computer languages allows the flow to occur from informal, qualitative descriptions to executable mathematically accurate representations in physics. In this case, the system architecture is modeled using SysML, which is connected to its physics-based model, which is established in Modelica. SysML can effectively represent the functional behavior and interfaces of the rover, and Modelica does the same for the physical behavior and interfaces in an executable manner, which is not possible when using natural language or informal diagrams or graphical representations. Collectively, both can be used to represent how signals can be passed to execute the physical response of the Rover based on the behavior in the systems model. This illustrates the explicit ability to unambiguously compare the verify and validate the Power and Drive Behavioral Specifications against a Physical representation of the product.

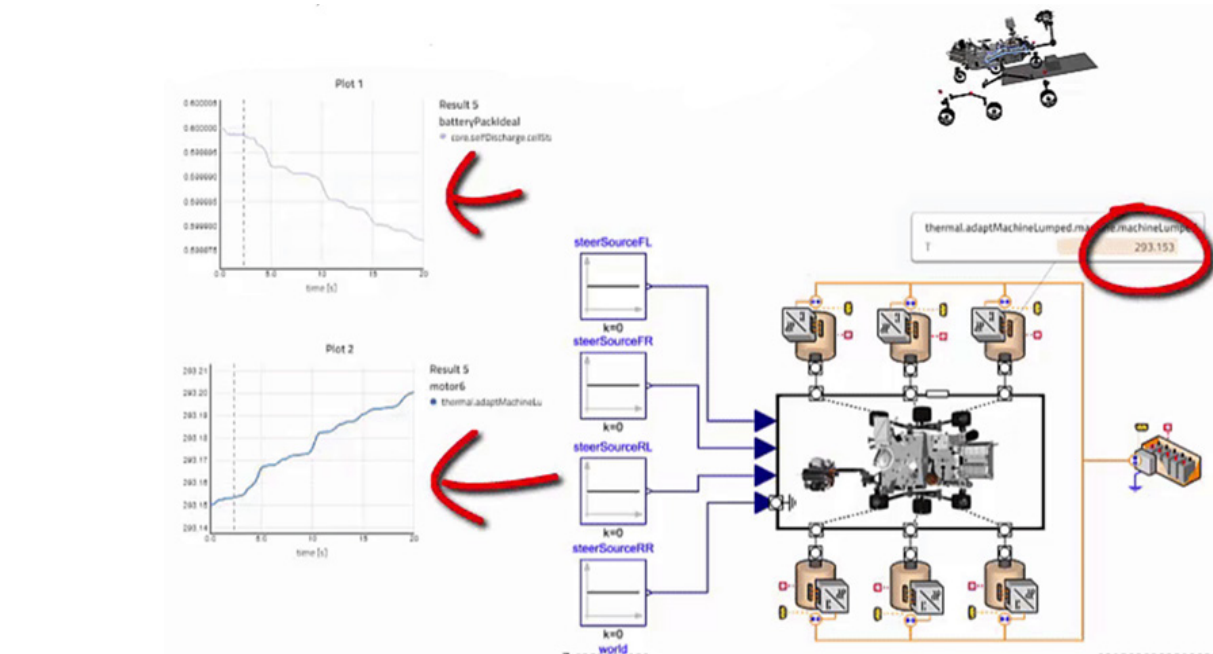


Figure 10. Iterative Phases of the Model-Based Development Process including Multi-Physics Simulation. Enabled by the Dragon Architecture, from [11]

4.5. Cognitive assistants

The developments in artificial intelligence and the computing power available today are enabling the development of cognitive assistants for systems engineering tasks. A cognitive assistant is a machine that acts as a virtual engineer, a senior companion with whom the human engineer works, and that performs engineering tasks that humans are not capable of carrying out effectively or efficiently.

With a cognitive assistant acting as a virtual engineer, the computer moves from being a tool in which we input data, create models, and run simulations, to a highly experienced work companion with whom we co-design, to whom we assign some of our tasks, on whom we rely to better understand what engineering decisions we should make, and who ensures that the knowledge generated in our project is stored within the organization so that other colleagues, and

ourselves, can benefit from it in future projects [28]. Instead of evaluating design options for a system, we will ask our virtual companion, *What do you think of this design?*, *Are we missing a key requirement?*, *What risk do you think we assume if we do not conduct this test?*, or *Would it cost us much to replace this interface with just a single command?*.

Cognitive assistants allow for the extraction and analysis of large amounts of data from various sources almost instantly and can provide the human engineer with the answers they need directly in natural language. Cognitive assistants can also initiate conversations unilaterally when they believe they can be useful to the human engineer, such as providing suggestions to improve an architecture that is currently being worked on. An example user interface is shown in Figure 11. By interpreting system models and relying on ontologies, cognitive assistants can improve their recommendations and performance when executing these functions.

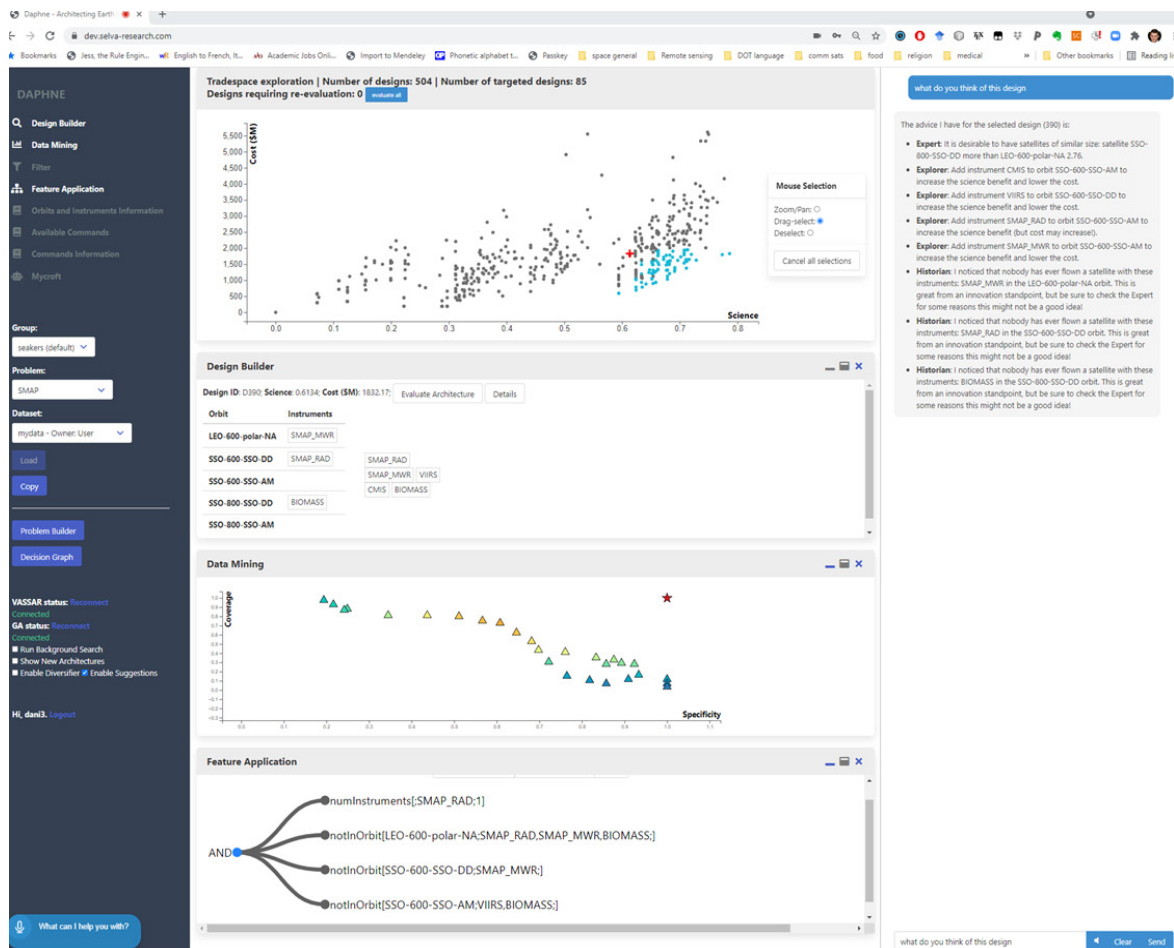


Figure 11. Daphne's user interface, a cognitive assistant to support the architecture of space systems [29]

5. CONCLUSIONS

This chapter has described how advances in computation and digital technologies provide organizations with the opportunity to digitalize (not just digitize) their processes in what is known as the 'Digital Transformation'. DE is about creating interoperable, rapidly accessible datasets that can be efficiently integrated across the systems lifecycle in what is known as the 'Digital Thread.' Declarative languages and Internet based interoperability provide the foundations to manage the data interoperability aspects, and open standards support the technical interoperability. Semantic web technologies can be leveraged to provide the framework for its implementation.

Throughout the chapter, multiple examples have been presented. The digital thread can provide unprecedented traceability across the lifecycle, is an enabler of early V&V, and supports model-based design review. Furthermore, DE gives rise to unprecedented systems engineering capabilities such as tradespace exploration, SBD, MDO, the integration of physics-based models early during system architecture, and even the adoption of cognitive assistants thanks to advances in AI. These all contribute to increasing the effectiveness and efficiency of system development.

REFERENCES

1. "DoD Instruction 5000.89: Test and Evaluation," 2020.
2. V. Singh and K. E. Willcoxy, "Engineering design with digital thread," AIAA/ASCE/AHS/ASC Struct. Struct. Dyn. Mater. Conf. 2018, pp. 1–21, 2018.
3. Z. Scott, "Speaking in Tongues: The Systems Engineering Challenge," INCOSE Int. Symp., vol. 29, no. 1, pp. 836–849, 2019.
4. P. De Saqui-Sannes, R. A. Vingerhoeds, C. Garion, and X. Thirioux, "A Taxonomy of MBSE Approaches by Languages, Tools and Methods," IEEE Access, vol. 10, no. October, pp. 120936–120950, 2022.
5. P. Witherell, S. Krishnamurty, I. R. Grosse, and J. C. Wileden, "Improved knowledge management through first-order logic in engineering design ontologies," Artif. Intell. Eng. Des. Anal. Manuf. AIEDAM, vol. 24, no. 2, pp. 245–257, 2010.
6. A. Patel and S. Jain, "Present and future of semantic web technologies: a research statement," Int. J. Comput. Appl., pp. 1–10, 2019.
7. N. Shadbolt, W. Hall, and T. Berners-Lee, "The semantic web revisited," IEEE Intell. Syst., vol. 21, no. 3, pp. 96–101, 2006.
8. D. Wagner, S. Y. Kim-Castet, A. Jimenez, M. Elaasar, N. Rouquette, and S. Jenkins, "CAESAR Model-Based Approach to Harness Design," in 2020 IEEE Aerospace Conference, 2020, pp. 1–13.
9. J. Gregory and A. Salado, "A Digital Engineering Factory for Students," in Conference on Systems Engineering Research (CSER), Tucson, AZ, USA, 2024.
10. OASIS Open Projects, "Open Services for Lifecycle Collaboration." 2023.
11. R. Karban et al., "Towards a Model-Based Product Development Process from Early Concepts to Engineering Implementation," in 2023 IEEE Aerospace Conference, 2023, pp. 1–18.
12. J. Gregory and A. Salado, "The Digital Engineering Factory: Considerations, Current Status, and Lessons Learned," in INCOSE International Symposium, Dublin, Ireland, 2024.
13. A. M. Madni and M. Sievers, "Model-based systems engineering: Motivation, current status, and research opportunities," Syst. Eng., vol. 21, no. 3, pp. 172–190, 2018.
14. R. Karban, F. G. Dekens, S. Herzig, M. Elaasar, and N. Jankevicius, "Creating system engineering products with executable models in a model-based engineering environment," Model. Syst. Eng. Proj. Manag. Astron. VI, vol. 9911, p. 99110B, 2016.
15. M. A. Bone, M. R. Blackburn, D. H. Rhodes, D. N. Cohen, and J. A. Guerrero, "Transforming systems engineering through digital engineering," J. Def. Model. Simul., vol. 16, no. 4, pp. 339–355, 2019.
16. K. Giammarco and K. Giles, "Verification and validation of behavior models using lightweight formal methods," Discip. Converg. Syst. Eng. Res., pp. 431–447, 2017.
17. R. Stevens, "Digital Twin for Spacecraft Concepts," IEEE Aerosp. Conf. Proc., vol. 2023-March, pp. 1–7, 2023.
18. W. K. Vaneman, R. Carlson, and C. Wolfgeher, "Defining a Model-Based Systems Engineering Approach for Milestone Technical Reviews," 2019.
19. M. R. Blackburn and B. Kruse, "Conducting Design Reviews in a Digital Engineering Environment," Insight, vol. 25, no. 4, pp. 42–46, 2022.
20. V. Romero, R. Pinquie, and F. Noel, "An Open Benchmark Exercise for Model-Based Design Reviews," in IFIP International Federation for Information Processing, 2023, pp. 176–185.
21. R. Pinquie, V. Romero, and F. Noel, "Survey of Model-Based Design Reviews: Practices & Challenges?," Proc. Des. Soc., vol. 2, pp. 1945–1954, 2022.
22. A. M. Ross and D. E. Hastings, "The tradespace exploration paradigm," 15th Annu. Int. Symp. Int. Counc. Syst. Eng. INCOSE 2005, vol. 2, pp. 1706–1718, 2005.
23. N. Shallcross, G. S. Parnell, E. Pohl, and E. Specking, "Set-based design: The state-of-practice and research opportunities," Syst. Eng., vol. 23, no. 5, pp. 557–578, 2020.
24. D. J. Singer, N. Doerry, and M. E. Buckley, "What is set-based design?," Nav. Eng. J., vol. 121, no. 4, pp. 31–43, 2009.
25. C. Small et al., "A UAV Case Study with Set-based Design," INCOSE Int. Symp., vol. 28, no. 1, pp. 1578–1591, 2018.
26. D. Raudberget, "Practical applications of set-based concurrent engineering in industry," Stroj. Vestnik/Journal Mech. Eng., vol. 56, no. 11, pp. 685–695, 2010.
27. H. Chen, "Multidisciplinary Design Optimization (MDO)," Encycl. Ocean Eng., no. July, 2021.
28. A. Salado and D. Selva, "Asistentes Cognitivos en Ingeniería de Sistemas," UEM STEAM Essentials, pp. 1–8, 2021.
29. A. Virós and D. Selva, "From design assistants to design peers: Turning daphne into an ai companion for mission designers," AIAA Scitech 2019 Forum, pp. 1–12, 2019.

Disclaimer/Acknowledgements

Part of the research presented in this chapter was carried out at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration (80NM0018D0004).

Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise, does not constitute or imply its endorsement by the United States Government or the Jet Propulsion Laboratory, California Institute of Technology.

CHRISTOPHER DELP

Chris Delp serves as the Technical Group Supervisor for Systems and Software Solutions Engineering, a group dedicated to implementing process-oriented methodologies for the creation of robust software environments designed to deliver advanced model based and digital engineering solutions. In addition, he holds the position of Manager of



the Computer Aided Engineering Systems Environment at NASA JPL. This effort is at the forefront of engineering innovation, providing a state-of-the-art environment for system modeling, analytical assessments, and simulation development grounded in Open Engineering principles. In his previous role, Mr. Delp spearheaded the Model Environment Development for the Europa Clipper mission, establishing a Model-Based Engineering Environment and fostering the growth of the Open MBEE community—a collective focused on open-source Model-Based Engineering models, software and frameworks. With a diverse background in Systems Engineering and Software Engineering on JPL Flight Projects, his expertise encompasses system architecture and design, the development and testing of safety-critical software, and the application of Systems Engineering principles throughout the project lifecycle. Mr. Delp is esteemed as a leading authority in Model-Based Systems Engineering (MBSE) and Model-Based Engineering Environments. He holds an MS in Systems Engineering from the University of Arizona.

DR. JOE GREGORY

Dr. Joe Gregory is a postdoctoral research associate at the University of Arizona. His research interests include digital engineering, model-based systems engineering, and the application of semantic web technologies to support engineering. In 2022, he received his PhD in Aerospace Engineering from the University of Bristol for his development of the SysML-based 'Spacecraft Early Analysis Model'. He is the co-chair of the Digital Engineering Information Exchange (DEIX) Taxonomy Working Group.

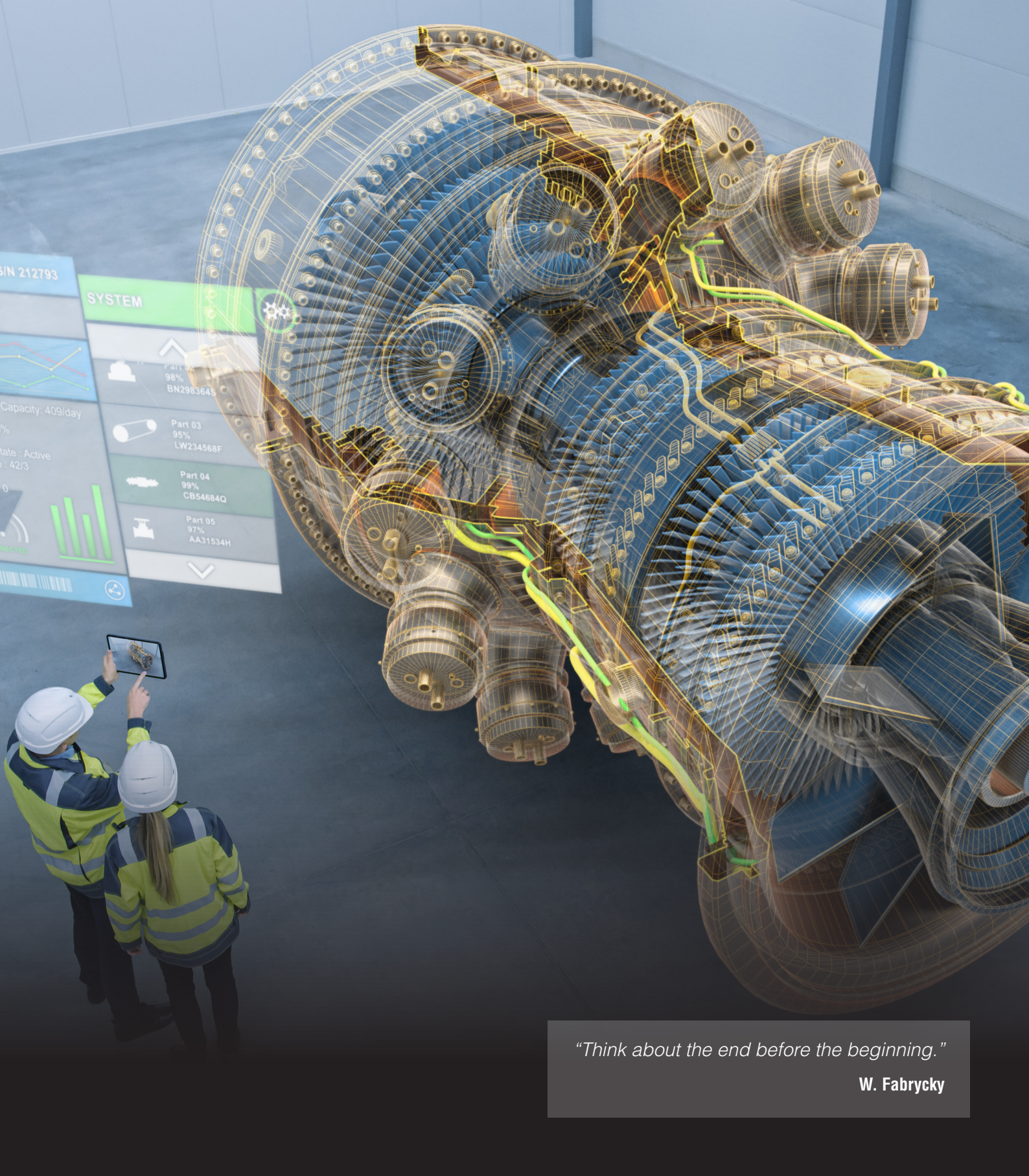


L. MIGUEL APARICIO

Luis Miguel Aparicio holds an BS/MS in Computer Science and Engineering from the Polytechnic University of Madrid and a graduate certificate in High Logistics Management from CESEDEN. He holds various certifications in logistics, systems engineering, and project management, including Lean Six Sigma, ITIL, INCOSE, PMP, and PRINCE.



After gaining professional experience at Coritel, Telefónica, and in the public sector, he joined Isdefe in 1999, where he remains to this day. He has collaborated as an adjunct professor at Rey Juan Carlos University and currently at the European University of Madrid. His professional experience has always revolved around information systems, with a particular focus on logistics. He currently serves as the Head of Logistics Systems Area at Isdefe, leading projects to support the rationalization and modernization of the logistical systems of the Spanish Armed Forces. He also has significant experience in projects related to the NATO Codification System, providing consultancy services in this field and supporting the development and modernization of SICAD, the Defense Cataloging System, a tool used in the National Cataloging Bureaus of Saudi Arabia, Belgium, Colombia, Peru, and Poland. Among the portfolio of projects managed within his area, there are several international consultancy projects, for example, with the European Defence Agency, NATO, the Military Industries Corporation of Saudi Arabia, or the National Cataloging Bureau of Jordan. He has also collaborated with the Armed Forces of Peru, the United Arab Emirates, Chile, and Ecuador. Currently, it is noteworthy the active role he plays in supporting the introduction of disruptive new technologies into Defense logistical information systems, with projects related to digital transformation, artificial intelligence, big data, or blockchain.



SIN 212793

SYSTEM

Capacity: 409/day

Status: Active

42/3

Part 02
98%
BN298364S

Part 03
95%
LW234568F

Part 04
98%
CB54684Q

Part 05
97%
AA31534H

"Think about the end before the beginning."

W. Fabrycky

Digital Transformation in System Deployment, Operation, Sustainment, and Retirement

Dr. Kaitlynn Castelle, *University of Maryland Applied Research Lab for Intelligence & Security* (kcastelle@arlis.umd.edu)

Miguel Ángel Coll Matamalas, *Isdefe* (macoll@isdefe.es)

Abstract

This chapter presents the novel capabilities enabled by Model-Based Systems Engineering (MBSE) and digital engineering to support system deployment, operations, sustainment, and retirement. Topics include advancements in the application of digital twin and digital thread technologies to support sustainment of systems through the life cycle and considerations for pursuing digital transformation and the adoption of advanced model-based technology. The chapter discusses the importance of planning for the use of these technologies early in the system life cycle to establish a foundation and strategy conducive to the application of model-based methodologies, automation, virtual reality, artificial intelligence, and other advanced technologies.

Keywords:

System Deployment, Operation, Sustainment, Retirement, Digital Twin, Digital Thread, Automation, Model Based Product Support (MBPS), Proactive Maintenance (CBM), Predictive Maintenance (PdM), Prescriptive Maintenance (RxM), Remaining Useful Life (RUL), Overhaul, Obsolescence Management, Configuration Management, Digital Transformation.



1. INTRODUCTION

Chapter 5 covered the use of digital engineering technology during the system development phase of the system life cycle. This chapter will dive into how a system's digital origin and foundation, and investments made in digital engineering technology enabling digital twin and digital thread, may be used through the later phases of the system's life cycle. The chapter also discusses considerations while pursuing this technology, addressing opportunities and risks through the deployment, operations and sustainment, and retirement of the system, including supporting logistics.

Interest in digital engineering technology, particularly those enabling digital twins and digital thread, has increased in recent years across various industries, specially manufacturing and defense. Three main concepts lay at the core of this interest: digital engineering, digital twin, and digital thread.

Digital engineering is a discipline that leverages advanced technologies, such as Computer-Aided Design (CAD), simulation, and data analytics, to support the design, development, and management of complex systems and products throughout their entire life cycle. It involves creating and integrating digital models, simulations, and data-driven decision-making to optimize design, manufacturing, deployment, operation, sustainment, and retirement processes, including logistics.

A **digital twin** is essentially a virtual representation of a physical object, system, or process (its physical twin) that enables real-time monitoring, analysis, and simulation. The concept of a digital twin allows organizations to have a deeper understanding of their assets, operations, and performance by connecting the physical and digital worlds through digital thread.

The term digital twin has been significantly abused though. While every digital twin will be a digital model, not every digital model necessarily constitutes a digital twin. Ideally, you could substitute a digital twin by its physical twin and vice versa, without noticing the change. This duality becomes very valuable because (1) you can experiment with the digital twin both in development phase and operational testing fairly inexpensively without risking its physical twin (the actual system) and (2) a digital model allows for accelerating discovery of system properties, that is, software simulation often runs faster than physical testing.

Figure 1 illustrates that there are many types of digital twins (virtual product twin, virtual process twin, digital factory twin and digital twin), depending on the phase of the product life cycle.

The **digital thread** refers to the means used for connecting the flow of data and information throughout the entire life cycle of a system or product. It enables effective communication and collaboration among various stakeholders across aspects and stages of the engineering life cycle, to ensure consistency and traceability across activities.

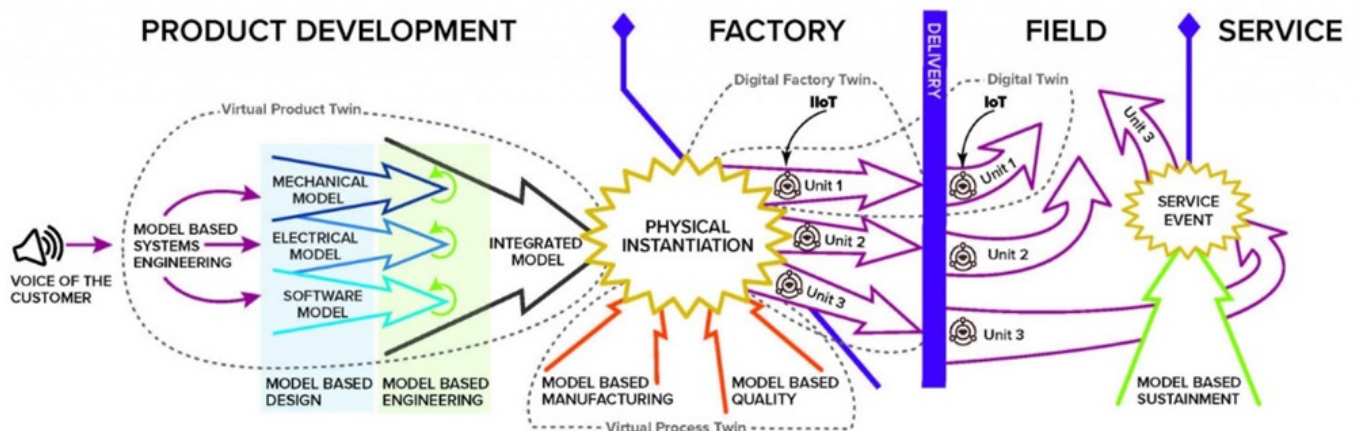


Figure 1. Digital Twins along the life cycle.

A more formal definition of digital thread, provided by the Defense Acquisition University (DAU), is as follows¹:

“An extensible and configurable analytical framework that seamlessly expedites the controlled interplay of technical data, software, information, and knowledge in the digital engineering ecosystem, based on the established requirements, architectures, formats, and rules for building digital models. It is used to inform decision makers throughout a system’s life cycle by providing the capability to access, integrate, and transform data into actionable information.”

Each organization’s digital transformation journey is unique and may support greater innovation, cost savings, and reliability in their products and systems. However, the technology and applications described in this chapter may not be appropriate or necessary for all organizations or systems of interest (Sol). In addition, many of the expected outcomes may not be achievable if the organization is not prepared to leverage the technology in certain contexts, or if the data models are not accessible to the system operating organization. Furthermore, it should be noted that similar to the application of systems engineering in physical systems, direct involvement of users, operators, maintainers, and other active stakeholders is necessary for the development of the digital twins. However, although the needs of the system designers and developers for the digital twin and digital thread may differ from the needs of the operator during the service phase, the use cases for later stages of the life cycle should be considered early in the system development life cycle.

2. APPLICATION OF DIGITAL TWIN, VIRTUAL ENVIRONMENTS, AND DIGITAL THREAD TECHNOLOGIES IN LATER SYSTEM LIFE CYCLE PHASES

2.1. Digitally transforming system deployment

Overall, digital engineering technology has expedited system deployments, reduced costs, increased agility, and improved the overall quality of deployed systems [1]. Below is a description and examples of some capabilities enabled by digital engineering and its applications during the entry into service.

2.1.1. Streamlined deployments with automation, virtualization, and hardware-in-the-loop testing

The use of virtualization technology allows for increased predictability in system deployments. Virtualization involves creating virtual versions of hardware, software, storage, or network resources, which may leverage emulation to mimic their behavior. In this way, systems can be deployed first in a virtual environment, reducing the need for excess physical infrastructure. This allows for faster and more efficient validation and deployment, as virtual systems can be quickly replicated and scaled as required [2].

Later, as the actual system is deployed, digital technologies enable the automation of deployment tasks, such as software installation, configuration, and testing. Automation decreases the chances of human error and allows for consistent and repeatable deployments. Furthermore, by facilitating the adoption of DevOps² practices (including continuous integration and deployment), some system changes can be automatically built, tested, and deployed to production environments, ensuring a faster and more reliable deployment process.

2.1.2. Quality control

Digital twins and virtual environments embed detailed geometric information that, coupled with associated metadata, helps ensure compliance with design specifications and standards, enabling better quality control during manufacturing and assembly. Detailed geometric information provides the foundation for the development of accurate simulations, analyses, and visualizations. It enables a comprehensive understanding of the physical entity’s shape, structure, and spatial relationships within the virtual environment, including additional information such as three-dimensional representation, dimensional accuracy, or material properties and textures, among others. Metadata associated with digital twins and virtual environments can be leveraged to enhance their understanding, management, and utilization, as it consists of descriptive information and properties that characterize and provide context to the digital representation of a physical object or system, such as life cycle information and accurate item identification (e.g., for configuration control).

1. Find this definition through the following link: <https://www.dau.edu/glossary/digital-thread> (last accessed on 23/04/2024).

2. DevOps is a software development methodology used as a set of practices and tools that integrates and automates the work of software development (Dev) and information technologies operations (Ops) as a means of improving and shortening the systems development life cycle.



Figure 2. Example of a Digital Twin (link: <https://youtu.be/G26mx4TnKyM?si=gR-fqlhmZw0B21-y>).

2.1.3. System training and familiarization

Digital environments generated through Augmented Reality (AR) or Virtual Reality (VR) yield immersive learning experiences that can greatly enhance system training and familiarization, enabling personnel to better understand and manage complex systems in a lifelike environment with less reliance on being in proximity of the physical asset (e.g., see Figure 2). When successful, these virtual systems can provide technicians and operators with real-time, on-site maintenance assistance utilizing AR overlays to communicate visual instructions and reference manuals directly on the physical system, resulting in heightened efficiency and accuracy during maintenance activities.

An example of such a method is product model visualization in a digital mock-up, which proves valuable when linking design and logistics supportability information to the geometric information embedded in digital twins. This connection, established by integrating technical and logistics data, creates a spatially aware context that improves comprehension of maintenance tasks [3]. Product model visualization is a concept that involves creating and presenting visual representations of physical objects or

systems, typically in a digital format. The primary goal is to convey information about the design, structure, and behavior of a product through graphical and interactive means. In the context of defense systems training and familiarization, for example, product model visualization plays a crucial role in providing an immersive and effective learning experience in a virtual environment. As an example, when coupled with the real system, an aviation maintainer can rely on digital artifacts directly projected and superimposed to the physical environment instead of relying on memorizing 2D diagrams and navigating dark, confined areas to locate specific parts [4]. Digital assets used in this way do not only improve training effectiveness and efficiency, but also actual task outcomes.

Despite their potential, challenges to adoption of these technologies remain, including but not limited to, ergonomic considerations and readily available and affordable VR/AR solutions [5]. As an intermediate alternative, digital artifacts can be provided to the operator or maintainer in the form of Interactive Electronic Technical Publications (IETP) or Manuals (IETM). These are digital formats of technical documents that provide comprehensive information, instructions, and guidance for the operation, maintenance, repair, and troubleshooting of complex systems or equipment [6]. These

publications are designed to replace or supplement traditional paper-based manuals with electronic versions that offer various interactive and multimedia elements to enhance user experience and facilitate efficient learning and comprehension. Compared to printed manuals, IETMs are becoming more and more widespread because they are interactive and easy to use and maintain, as well as provide insight into the system that would otherwise be difficult to infer.

2.2. Digitally transforming operations and sustainment

2.2.1. Model Based Product Support and configuration management

Model Based Product Support (MBPS) consists of applying the principles and concepts of digital engineering to Integrated Life Cycle Support (ILS). In that sense, it refers to the modelling of any aspect related to ILS, including its supportability, reliability, and safety characteristics, both in terms of design and life cycle processes. MBPS leverages digital models, simulations, the interoperability of information systems and data to enhance the efficiency and effectiveness of supporting systems to improve operational readiness and to optimize sustainment and life cycle management throughout their useful life. Particularly, system models generated during system development are used during the operation and servicing phase to help understand the system functionality and its external and internal interfaces.

Data continuity, which is essential to support a coherent transition between the models generated and/or used in system development and those generated and/or used in MBPS, is enabled by the digital thread. On the other hand, maintaining an accurate configuration baseline of an asset or system remains critical in order to ensure it can be logistically supported. Applying MBPS by using authoritative sources of truth, it becomes no longer necessary to manually maintain two different configurations, one for design and one for logistics, throughout the life cycle, by incorporating all relevant logistics information directly onto the configured design elements themselves.

2.2.2. Proactive maintenance strategies enabled by digital twins and digital thread

Traditionally, both corrective and preventive maintenance have been the most widely used strategies of maintenance. In corrective maintenance, repairs are carried out after the problem has occurred. On the contrary, preventive maintenance focuses on preventing potential failures before they occur by executing scheduled maintenance actions, such as inspections, routine servicing, and component replacements (generally based on use patterns: running hours, cycles, number of starts, etc.). However, advancements in sensors and monitoring technologies and the resulting development of digital twins and the digital thread enable a turn towards data-driven maintenance approaches. These approaches rely on the use of algorithms to make predictions and recommendations to optimize system's reliability and the operational efficiency of the assets.

By moving from reactive to data-driven maintenance, organizations can minimize asset downtime and reduce maintenance costs, since potential failures modes can be detected and addressed before they occur, preventing operation interruptions and emergency repairs. Three main proactive maintenance strategies can be distinguished:

- **Condition Based Maintenance (CBM).** CBM focuses on monitoring the real-time condition of equipment by using various sensors and data collection tools to assess the current state of the equipment. CBM uses thresholds, or set conditions, to trigger alarms or maintenance actions when deviations from normal operating conditions are detected.

CBM typically focuses on a specific set of parameters relevant to the equipment being monitored, and continuous monitoring of a few critical factors that directly impact immediate performance or reliability. The primary objective of CBM is to monitor the current condition of equipment and take timely actions to prevent imminent failures or issues.

CBM is used in a broad variety of applications, including manufacturing, defense, industrial control systems, healthcare, to diagnose impending failure modes, thereby reducing maintenance costs, improving reliability, availability, and safety, extending time between overhauls (compared with traditional preventive maintenance), and reducing unnecessary downtime. With the ability to continuously monitor the asset's health and performance metrics, system data can be collected to develop models enabling proactive detection of failure modes.

- **Predictive Maintenance (PdM).** PdM also involves monitoring equipment condition, but it primarily focuses on using real time data analytics and historical data to predict when maintenance action is needed. PdM analyzes patterns and trends in data to forecast potential.

Although CBM and PdM share similarities and often use similar technologies, there are some distinctions in them: CBM initiates maintenance actions based on the real-time condition of the equipment and focused on a specific set of parameters, while PdM aims to schedule maintenance activities just before the equipment is likely to fail, optimizing maintenance timing to prevent failures and minimize downtime.

PdM often involves more comprehensive data analysis than CBM, using a broader range of data points and indicators to predict failures. It may incorporate advanced analytics, Artificial Intelligence algorithms, like machine learning, and other sophisticated algorithms to forecast failures, while CBM uses them less intensively.

- **Prescriptive Maintenance (RxM).** Prescriptive maintenance goes beyond PdM. It does not only predict potential equipment failures but also prescribes specific actions to prevent those failures or mitigate their impact. This approach integrates predictive analytics with automated decision-making systems to provide precise recommendations or prescriptions for maintenance actions. These recommendations can include detailed instructions on maintenance tasks, repairs, adjustments, or operational changes. RxM utilizes sophisticated algorithms of artificial intelligence and Big Data techniques to analyze extensive sets of data. It examines historical and real-time data to predict potential failures and determines the best course of action to avoid or address these issues. In the context of RxM, prognosis plays a significant role in determining the appropriate actions to be recommended. Prognosis refers to the estimation or prediction of future conditions or events based on the analysis of current and historical data. In RxM, prognosis involves forecasting potential failures, estimating the Remaining Useful Life (RUL), and predicting the future health and performance of assets or equipment [6].

Digital twins become central across maintenance approaches thanks to their capabilities to enable data-driven applications: real-time monitoring and analysis, predictive and prescriptive analytics, simulating “what-if” scenarios, health monitoring and prognosis, and optimization analysis. In predictive maintenance, digital twins continuously collect real-time data from sensors embedded in the physical assets and utilize

historical data to analyze trends, patterns, and anomalies. By leveraging this information, digital twins facilitate predictive analytics and forecasting of potential failures or performance degradation. In prescriptive maintenance, digital twins play a crucial role, by providing not only predictive insights, but also actionable recommendations. By simulating different scenarios and analyzing data, digital twins can suggest specific maintenance actions or strategies to prevent failures or optimize asset performance. They also aid in prescribing the most effective actions to be taken based on predictive analytics and simulations.

Finally, it should be noted that digital twins can also be of value to support preventive maintenance, since they can assist in establishing asset's baseline conditions, continuously monitoring asset conditions, and identifying deviations from normal operating parameters by tracking equipment health.

2.2.3. Modernizing Maintenance, Repair and Overhaul (MRO)

As assets age, there may be a need for component replacements, upgrades, or redesigns. The geometric information embedded in digital twins, along with metadata on materials and manufacturing tolerances, aids in identifying compatible replacement parts and planning seamless upgrades. This can help in anticipating the integrability of new components with the existing system, avoiding compatibility issues and potential disruptions.

Integrating CAD enables effective collaboration among stakeholders, seamlessly incorporating modifications from an asset's life cycle into the CAD model for future reference. CAD models created during the design phase become the basis to provide a visual representation for the physical asset in the digital twin. This digital representation, with precise geometric data, later allows maintenance teams to visualize and understand the asset's components and assemblies, aiding in troubleshooting and planning maintenance activities, and additionally supporting communication of possible configuration changes to the deployed asset.

Alongside the geometric representation, digital twins incorporate essential metadata, such as material properties, manufacturing tolerances, and part numbers. These metadata provide crucial information for manufacturing, assembly, and future maintenance activities. Particularly, it aids maintenance personnel in quickly identifying the right components, accessing relevant information, and performing